



Universidad
de Concepción

Sistemas Operativos 2026-1

INFORME TAREA 1: "Implementación de una Shell Simple"

Matrícula	Nombre Completo	Docentes	Repositorio Github
2025423669	Lucas Fierro Orellana	Juan Felipe Gonzáles Saavedra Cecilia Paola Hernández Rivas	<u>LINK</u>
2025432692	Eduardo Riveros Medina		
2025431734	Vicente Vergara Lizama		
2025404117	Matias Cabello Carvajal		

Descripción general Shell:

Implementamos una shell basada en UNIX/POSIX bajo un ciclo de Lectura-Evaluación-Impresión, con manejo robusto de señales. Su flujo principal es:

1. Limpieza asincrónica: Antes del prompt, `jobs_avisar_terminados()` limpia y notifica sobre procesos en background finalizados.
2. Lectura y parseo: Lee la entrada con `getline()` y detecta el `&` (background). En lugar de `strtok`, un autómata separa los argumentos respetando los espacios dentro de comillas. Si detecta tuberías `|`, deriva la subdivisión a "pipes.c"
3. Ejecución y Descriptores: Verifica primero si es un comando built-in para ser ejecutado en el proceso padre. Si es externo, procesa las redirecciones de E/S (`<`, `>`, `>>`), hace un `fork()`, restaura las señales y desvía flujos con "`dup2()`" antes de llamar a "`execvp()`". El proceso padre decide si se espera al hijo o se registra asincrónicamente en la tabla de jobs.

Estructura de datos para Jobs:

Para gestionar el background se usó un arreglo estático `static Job tabla_jobs [MAX_JOBS]` con un límite de 64 trabajos simultáneos, partido en 0 al iniciar la Shell.

El ciclo de vida de cada job se controla mediante el enum `JobEstado` (`JOB_EJECUTANDO`, `JOB_TERMINADO`, `JOB_DETENIDO`). A su vez, cada comando en background se modela con un struct `Job` que empaqueta la siguiente información: ID, PIDs de todos los procesos, ID del grupo de procesos, cantidad de PIDs, el texto del comando visible, su estado, flag de espacio ocupado y un flag de completado.

Implementación de Tuberías (Pipes de largo arbitrario):

Para el manejo de múltiples procesos conectados con N comandos, la lógica reside dentro de la función `ejecutar_tuberias`. Donde tomamos un enfoque de puente/túnel que arrastra el descriptor de lectura del comando anterior a través de una variable llamada `fd_in`. Se itera con `total_cmds` veces:

1. En cada iteración, se crea un nuevo pipe(`fd`), a excepción del último.
2. Hacemos `fork()`
3. Dentro del proceso hijo, si `fd_in` tiene datos de un pipe previo, se conecta a STDIN con `dup2`. Luego, si no es el último comando, se conecta a STDOUT a la entrada del pipe actual `fd[1]`. Finalmente cerramos los descriptores y hacemos `execvp()`
4. Dentro del proceso padre, cerramos los descriptores que no usamos. Actualizamos `fd_in = fd[0]` para que el siguiente comando lea de allí. El cierre minucioso de los descriptores evita deadlocks por esperar un EOF.

Manejo de Señales (SIGCHLD, SIGINT):

Durante la inicialización, la shell principal utiliza `sigaction()` con `SIG_IGN` para ignorar señales interruptivas como `SIGINT` (Ctrl+C), `SIGQUIT` y `SIGTSTP` (Ctrl+Z), asegurando su persistencia. Al realizar un `fork()`, los procesos hijos reestablecen estas disposiciones a su estado original (`SIG_DFL`) antes del `execvp()`, permitiendo que el usuario detenga o interrumpa los comandos lanzados.

Recolección en Background (`SIGCHLD`): Se instaló un handler asíncrono para notificar y limpiar a los procesos terminados de forma silenciosa. El handler utiliza `waitpid(-1, &status, WNOHANG)` dentro de un ciclo while.

Esto previene la aparición de procesos zombies sin bloquear la lectura interactiva de la shell, iterando hasta recolectar todos los hijos que hayan finalizado de manera concurrente.

Diseño del comando de monitoreo pmon:

La función de monitoreo, "jobs_pmon" se mantiene activa utilizando pause(), que se despierta con alarmas generadas con alarm(segundos). Estas interrupciones activan banderas declaradas bajo volatile sig_atomic_t para prevenir condiciones de run durante la lectura/escritura de estados de señal.

Extracción de datos: La función itera sobre los jobs activos. Abre manualmente /proc/[pid]/stat omitiendo campos con sscanf() hasta capturar el estado (State) y los valores de utime y stime. También escanea /proc/[pid]/status para obtener la memoria en VmRSS.

Guarda el sumatorio temporal previo de los ticks en un arreglo de memoria volátil (MuestraCPU). Calcula la diferencia temporal entre el refresco actual y el anterior, luego la convierte a segundos usando la constante POSIX sysconf(_SC_CLK_TCK). Esta proporción luego se traduce al porcentaje visualizado.

Funcionalidad Adicionales:

Se implementó soporte completo para Ctrl+Z (SIGTSTP), junto con los comandos internos fg y bg. Para lograrlo, la shell crea su propio grupo de procesos y toma el control del terminal (tcsetpgrp()). Todo nuevo comando (o tubería) se aísla en su propio grupo (setpgid). Si un proceso en foreground recibe un Ctrl+Z, la shell recupera el terminal y lo registra en la tabla interna con el estado JOB_DETENIDO. Los comandos fg y bg utilizan el ID del job para recuperar su grupo de procesos original, devolverle el control del terminal (en el caso de fg) y reanudar su ejecución enviándole la señal SIGCONT mediante la función kill().

Ordenamiento en Monitor: Se incorporó una rutina de recolección y ordenamiento dinámico con Bubble Sort en la visualización de pmon. Esto permite ordenar la lista de procesos según el consumo de CPU (mayor a menor, similar a top), resaltando con rojo al proceso que registre con mayor impacto en el sistema.

Preguntas de Reflexión:

1. ¿Por qué cd y exit deben implementarse como comandos internos y no mediante fork() y exec()?

Cada proceso dentro de Linux maneja un propio entorno con su propio directorio de trabajo, si el cd se realizara a través de fork() y exec(), sería el proceso hijo quién cambiaría ruta. Al finalizar su proceso, la shell principal (proceso padre), quedaría en el directorio original, dejando inútil el propósito del comando. De esta forma, si exit se ejecutara en un proceso hijo, solo terminaría ese proceso y la shell seguiría en ejecución. Por esto mismo, debe ser un built-in para alterar el estado y ciclo de vida del proceso principal

2. En una tubería cmd1 | cmd2 | cmd3, ¿qué pasaría si no se cerraran los descriptores de pipe no usados en cada proceso? Describa un escenario concreto de falla.

Si no se cierran los descriptores de escritura de los pipes no utilizados, los procesos que actúan como lectores nunca reciben la señal de Fin de archivo (eof). El sistema operativo mantendrá abierto el canal asumiendo que existe un proceso que aún puede enviar datos (incluyendo la propia shell).

Como escenario de falla, Al ejecutar un "ls | wc -l", el comando ls escribe sus datos en el pipe y luego termina. Sin embargo, si dentro de esto, la shell no cierra su copia del descriptor de escritura asociado a este pipe, el proceso wc -l, se quedará bloqueado de manera indefinida esperando datos. Como consecuencia, la shell (con su waitpid al proceso hijo) también queda esperando.

3. ¿Por qué el manejador de SIGCHLD debe llamar a waitpid() en un ciclo con WNOHANG, en vez de una sola vez?

Cuando varios hijos terminan casi al mismo tiempo el sistema envía un solo aviso en una señal, así, si el manejador llama a `waitpid()` una sola vez, se recoge a un solo hijo dejando al resto como procesos zombies porque no habría más avisos para recogerlos. Se debe usar un ciclo con `WNOHANG` porque sin él, `waitpid()` espera a que termine un hijo. Si no quedaran más hijos se quedaría esperando sin fin. Con `WNOHANG` `waitpid()` recoge los hijos que hayan terminado y retorna 0 cuando no queda ninguno.

4. ¿Qué diferencia de comportamiento observarían si, en vez de restaurar SIGINT a su disposición por defecto en el hijo, simplemente no tocan la disposición heredada de la shell?

Se generaría la imposibilidad de detener los comandos que se están ejecutando con Ctrl+C.

El proceso padre debe ignorar la señal "sigint" para evitar que se cierre al cancelar una línea de texto. Ahora, el problema queda generado por la llamada `fork()`, que clona el estado del padre al hijo, incluyendo la disposición a esa señal, o sea, si no se restaura la señal a la disposición por defecto (`SIG_DFL`), el usuario podría ejecutar un comando largo o que quede esperando una entrada, como `cat` (infinito por ausencia de argumento) o `ping` (infinito por paquetes), y no se podría detener el proceso (al no tener disponible el `ctrl+c`), obligando al usuario a tener que usar manualmente "kill".

5. En pmon, ¿qué ocurre si un proceso monitoreado termina justo entre dos lecturas de /proc? ¿Cómo lo manejó su implementación?

Si un proceso monitoreado termina entre dos lecturas de `/proc`, su directorio desaparecerá. Esto se logra con estas 3 implementaciones:

- 1.- Valida archivos nulos (leer `proc stat`): La función intenta abrir el archivo con `fopen(ruta, "r")`. Si ya se eliminó el directorio del proceso, `fopen` devuelve null y como se tiene el `if (f == NULL)`, se retorna -1 que evita lectura de buffers vacíos y errores de segmentación.
- 2.- `jobs_pmon`: Dentro del ciclo que calcula el uso de CPU, solo se imprimen si entre en el `if (leer_proc_stat(pid, &state, &ticks_actual) == 0)` y como el código, por el mecanismo del punto 1, recibe -1, se salta el PID y se evita mostrar información que no corresponde.
- 3.- Limpieza y sincronización: Cada vez que el temporizador actualiza nuevamente "pmon_flag_alarma" a 1, la primera instrucción que se ejecutará será "`jobs_avisar_terminados()`", asegurándose que los procesos muertos pasen por el garbage collector, siendo recolectados del estado zombie y desmarcando su estado de activo en "tabla jobs", logrando que se asegure que en las siguientes iteraciones ya no se vuelva a intentar leer.

Pruebas Realizadas:

Para validar nuestra shell, se hizo un script de pruebas de estrés (`test_extremo.sh`), este revisa el comportamiento frente a: comillas y espacios continuos, pipes largos con redirección de E/S, transiciones dinámicas de control de procesos (`bg`, `fg`, `CTRL+Z`) y manejo de fallos críticos. Todo validado con Valgrind (0 bytes in 0 blocks in use at exit) en padres e hijos.

Limites Conocidos de la Shell:

- La tabla global soporta un máximo de 64 trabajos en segundo plano simultáneos (`MAX_JOBS`).
- Los pipes agrupados en un mismo trabajo tienen un límite de 16 procesos hijos (`MAX_PIDS_POR_JOB`)
- Aunque la shell ejecuta comandos de largo infinito con `getline()`, el texto visible para job tiene un límite de 1023 caracteres (`cmdline[1024]`).
- La visualización de la ruta de trabajo tiene un máximo de 1024 caracteres para el buffer `cwd`.